Advanced Database - Assignment 1: Data Warehouse

- **Group ID:** **DS – G9**

- **DB Source Name:** **EPL 21-22 English Premier League**

- **Group names and IDs:**

- **Haneen Hisham 20236032**
- **Shaza Moatasem 20236050**
- **Rabab Mohamed 20237004**
- **Ziad Tarek 20236043**

- **Emails:**

- **Haneen** 20236032@stud.fci-cu.edu.eg
- **Shaza** 20236050@stud.fci-cu.edu.eg
- **Rabab** 20237004@stud.fci-cu.edu.eg
- **Ziad** 20236043@stud.fci-cu.edu.eg

1. Motivation and Business Questions

The goal of this project is to build a small data warehouse for the English Premier League 2021–2022 season that supports analytical queries for coaches, analysts and fans. Instead of repeatedly querying the raw CSV files, the data warehouse organizes the data into a dimensional model optimized for analysis.

Example questions that our data warehouse is designed to answer include:

- Who are the top goal scorers in the league?
- How do teams perform at home compared to away matches?
- Which playing positions (forward, midfielder, defender, goalkeeper) contribute the most goals?
- Which matches were the highest scoring?
- How do team statistics in the league table relate to player-level performance?

The dimensional model and ETL processes described in the following sections are built specifically to support these types of queries.

2. Source System (OLTP Model)

The operational database for this project consists of three main tables imported from the English Premier League 2021–2022 dataset:

points_table:

- Stores league standings for each team including matches played, wins, draws, losses, goals for/against, goal difference, and total points.
- Primary key: Team

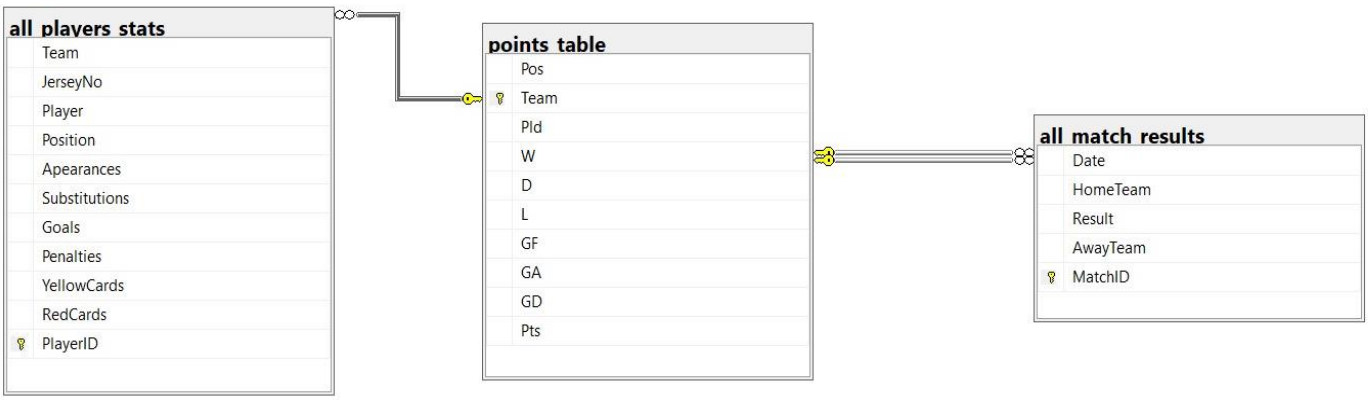
all_match_results:

- Contains match-level data including match date, home team, away team, and the final score in TIME format (e.g. 02:01:00 represents a 2–1 score).
- Primary key: MatchID (added during preprocessing)
- Foreign keys: HomeTeam → points_table.Team
  AwayTeam → points_table.Team

all_players_stats:

- Contains individual player statistics for the season such as appearances, substitutions, goals, penalties, yellow/red cards.
- Primary key: PlayerID (added during preprocessing)
- Foreign key: Team → points_table.Team

(Figure 1 – Source Database OLTP ERD)



The **ERD** shows the structure of the source operational database used for the data warehouse. The **points_table** stores team-level performance statistics and acts as the central reference table. It is linked to **all_players_stats**, which contains individual player statistics, and **all_match_results**, which records match outcomes. Each team can have multiple players and appear in multiple matches as home or away, forming one-to-many relationships. This relational model enables accurate data extraction and transformation into the dimensional data warehouse.

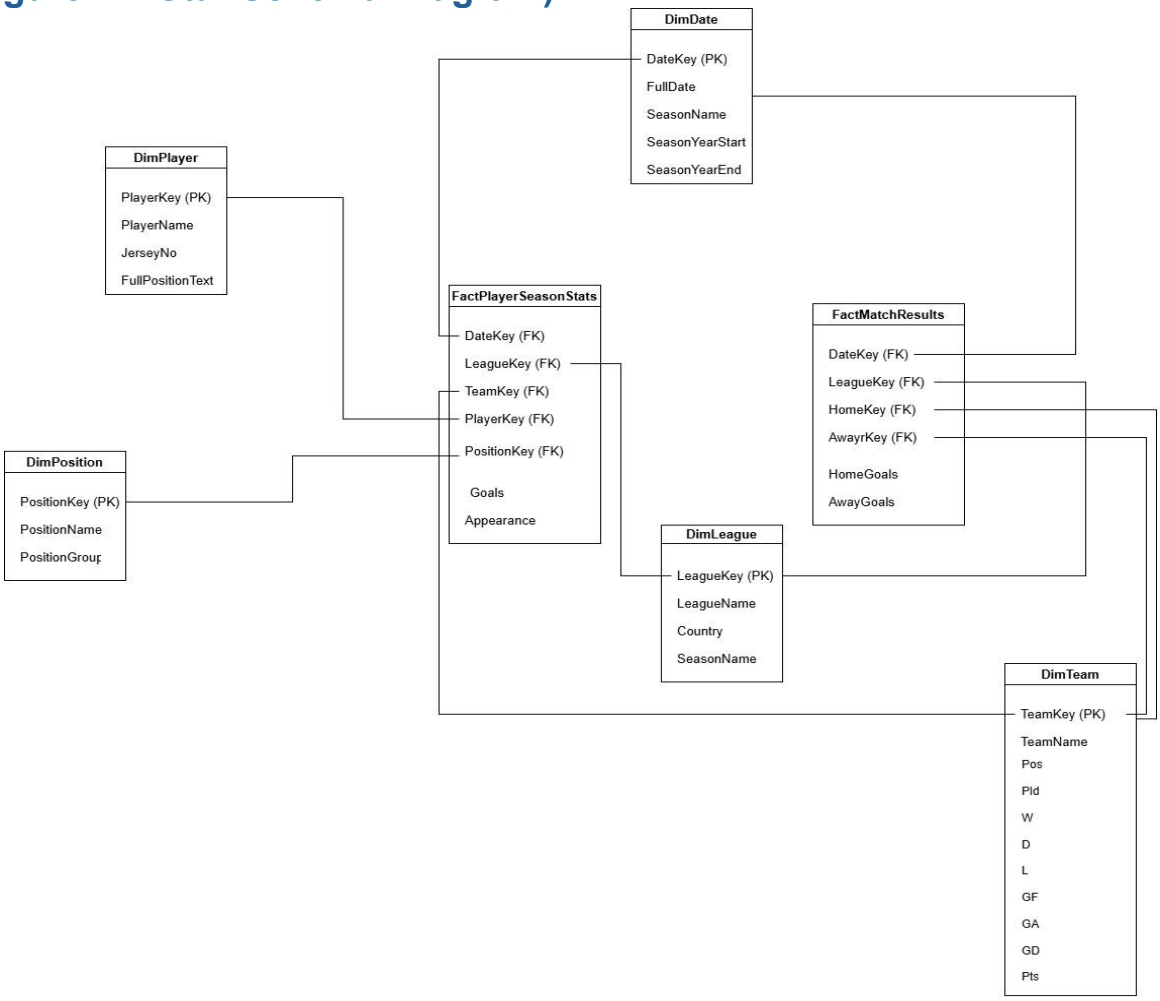
3. Star Schema Overview

The data warehouse for the EPL 2021–2022 season is designed using a **star schema** to support analytical reporting related to player performance, team standings, and match results. The schema consists of **two fact tables** connected to **five dimension tables**, enabling multidimensional analysis.

The conceptual model is optimized for analytical queries and includes:

- A **Date dimension** representing the football season
- A **League dimension** describing competition context
- **Team, Player, and Position dimensions** for analytics at different entity levels
- **Measures** such as *Goals, Appearances, HomeGoals*, and *AwayGoals* forming the core of the analytical layer

(Figure 2 – Star Schema Diagram)



3.1 Dimension Tables

1. DimDate

- **Grain:** One row per football season (not per calendar day)
- **Primary Key:** DateKey
- **Attributes:** FullDate, SeasonName, SeasonYearStart, SeasonYearEnd
- **Hierarchy:** No lower-level time hierarchy (e.g., month/day). This dimension operates at the season level only.

2. DimLeague

- **Grain:** One row per league per season
- **Primary Key:** LeagueKey
- **Attributes:** LeagueName, Country, SeasonName
- **Hierarchy:** Flat (no multi-level hierarchy)

3. DimTeam

- **Grain:** One row per team in the season
- **Primary Key:** TeamKey
- **Attributes:**
  TeamName, Pos, Pld, W, D, L, GF, GA, GD, Pts
- **Source:** Loaded from points_table
- **Hierarchy:** Flat structure (no sub-levels)

4. DimPlayer

- **Grain:** One row per player
- **Primary Key:** PlayerKey
- **Attributes:** PlayerName, JerseyNo, FullPositionText
- **Used by:** FactPlayerSeasonStats

5. DimPosition

- **Grain:** One row per position
- **Primary Key:** PositionKey

- **Attributes:** PositionName, PositionGroup
- *Example*: FW (Forward), MF (Midfielder), DF (Defender), GK (Goalkeeper)
- **Hierarchy:** Flat structure; optional grouping via PositionGroup
- **Used by:** FactPlayerSeasonStats

3.2 Fact Tables

1. FactPlayerSeasonStats

- **Grain:** One row per *player–position–team–league–season*
- **Foreign Keys:** DateKey, LeagueKey, TeamKey, PlayerKey, PositionKey
- **Measures:**
  - Goals
  - Appearances
- Enables analytics such as top goal scorers, performance breakdown by team or position.

2. FactMatchResults

- **Grain:** One row per match
- **Foreign Keys:** DateKey, LeagueKey, HomeTeamKey, AwayTeamKey
- **Measures:**
  - HomeGoals
  - AwayGoals
- Enables analytics such as highest scoring matches, home vs away team performance, and competitive comparisons.

The match score was originally stored as a TIME value (e.g., 02:01:00). It was transformed into numeric goal values using T-SQL logic during ETL:

```
CAST(LEFT(CONVERT(VARCHAR(8), Result, 108), 2) AS INT) AS HomeGoals,
CAST(SUBSTRING(CONVERT(VARCHAR(8), Result, 108), 4, 2) AS INT) AS AwayGoals
```

4. Analytical Queries on the Data Warehouse

This section presents example analytical queries that can be answered using the data warehouse star schema. All queries use the dimensional model tables (dw. schema), not the raw source CSV tables.

Query 1 – Top 10 Goal Scorers

Business question:

Who are the top 10 goal scorers in the league, and which teams do they play for?

SQL:

```sql
SELECT TOP 10
    p.PlayerName,
    t.TeamName,
    f.Goals,
     f.Appearances
FROM dw.FactPlayerSeasonStats AS f
JOIN dw.DimPlayer AS p ON f.PlayerKey = p.PlayerKey
JOIN dw.DimTeam   AS t ON f.TeamKey   = t.TeamKey
ORDER BY
    f.Goals DESC;
```

Result:

100 %

Results | Messages

	PlayerName	TeamName	Goals	Appearances
1	Mohamed Salah	Liverpool	31	44
2	Harry Kane	Tottenham Hotspur	27	47
3	SON Heung-min	Tottenham Hotspur	24	39
4	Riyad Mahrez	Manchester City	24	31
5	Cristiano Ronaldo	Manchester United	24	35
6	Sadio Mané	Liverpool	23	45
7	Diogo Jota	Liverpool	21	39
8	Kevin De Bruyne	Manchester City	19	39
9	Jarrod Bowen	West Ham United	18	45
10	James Maddison	Leicester City	18	40

Query 2 – Goals by Playing Position

Business question:

Which positions (e.g., FW, MF, DF) contribute the most goals?

SQL:

```sql
SELECT
    pos.PositionName,
    SUM(f.Goals) AS TotalGoals
FROM dw.FactPlayerSeasonStats AS f
JOIN dw.DimPosition AS pos
    ON f.PositionKey = pos.PositionKey
GROUP BY
    pos.PositionName
ORDER BY
    TotalGoals DESC;
```

Result:

100 %

Results | Messages

	PositionName	TotalGoals
1	Midfielder/Forward	491
2	Forward	338
3	Midfielder	315
4	Defender	130
5	Defender/Midfielder	114
6	Defender/Forward	9
7	Defender/Midfielder/Forward	2
8	Goalkeeper	0

Query 3 – Team Goals For and Against

Business question:

For each team, how many goals did they score and concede during the season?

SQL:

```
SELECT t.TeamName,
    SUM(CASE
            WHEN f.HomeTeamKey = t.TeamKey THEN f.HomeGoals
            WHEN f.AwayTeamKey = t.TeamKey THEN f.AwayGoals
            ELSE 0
        END) AS GoalsFor,
    SUM(CASE
            WHEN f.HomeTeamKey = t.TeamKey THEN f.AwayGoals
            WHEN f.AwayTeamKey = t.TeamKey THEN f.HomeGoals
            ELSE 0
        END) AS GoalsAgainst
FROM dw.DimTeam AS t
JOIN dw.FactMatchResults AS f ON f.HomeTeamKey = t.TeamKey OR
f.AwayTeamKey = t.TeamKey
GROUP BY t.TeamName
ORDER BY GoalsFor DESC;
```

Result:

Results | Messages

	TeamName	GoalsFor	GoalsAgainst
1	Manchester City	99	26
2	Liverpool	94	26
3	Chelsea	76	33
4	Tottenham Hotspur	69	40
5	Leicester City	62	59
6	Arsenal	61	48
7	West Ham United	60	51
8	Manchester United	57	57
9	Aston Villa	52	54
10	Crystal Palace	50	46
11	Brentford	48	56
12	Newcastle United	44	62
13	Everton	43	66
14	Southampton	43	67
15	Leeds United	42	79
16	Brighton and Hov...	42	44
17	Wolverhampton ...	38	43
18	Burnley	34	53
19	Watford	34	77
20	Norwich City	23	84

Query 4 – Highest Scoring Matches

Business question:

Which matches had the highest combined number of goals?

SQL:

```
SELECT TOP 10
    ht.TeamName AS HomeTeam,
    at.TeamName AS AwayTeam,
    f.HomeGoals,
    f.AwayGoals,
    (f.HomeGoals + f.AwayGoals) AS TotalGoals
FROM dw.FactMatchResults AS f
JOIN dw.DimTeam AS ht
    ON f.HomeTeamKey = ht.TeamKey
JOIN dw.DimTeam AS at
    ON f.AwayTeamKey = at.TeamKey
ORDER BY
    TotalGoals DESC;
```

Result:

Results | Messages

	HomeTeam	AwayTeam	HomeGoals	AwayGoals	TotalGoals
1	Manchester City	Leicester City	6	3	9
2	Manchester City	Leeds United	7	0	7
3	Chelsea	Norwich City	7	0	7
4	Everton	Watford	2	5	7
5	Leicester City	Manchester United	4	2	6
6	Burnley	Crystal Palace	3	3	6
7	Newcastle United	Brentford	3	3	6
8	Leicester City	Watford	4	2	6
9	Manchester United	Leeds United	5	1	6
10	Newcastle United	West Ham United	2	4	6

Query 5 – Home vs Away Goals per Team

Business question:

How do teams perform at home compared to away in terms of goals scored?

SQL:

```
SELECT
    t.TeamName,
    SUM(CASE WHEN f.HomeTeamKey = t.TeamKey THEN f.HomeGoals ELSE 0
END) AS HomeGoals,
    SUM(CASE WHEN f.AwayTeamKey = t.TeamKey THEN f.AwayGoals ELSE 0
END) AS AwayGoals
FROM dw.FactMatchResults AS f
JOIN dw.DimTeam AS t
    ON t.TeamKey = f.HomeTeamKey
    OR t.TeamKey = f.AwayTeamKey
GROUP BY t.TeamName
ORDER BY HomeGoals DESC;
```

Result:

Results | Messages

	TeamName	HomeGoals	AwayGoals
1	Manchester City	58	41
2	Liverpool	49	45
3	Tottenham Hotspur	38	31
4	Chelsea	37	39
5	Arsenal	35	26
6	Leicester City	34	28
7	West Ham United	33	27
8	Manchester United	32	25
9	Aston Villa	29	23
10	Crystal Palace	27	23
11	Everton	27	16
12	Newcastle United	26	18
13	Southampton	23	20
14	Brentford	22	26
15	Wolverhampton Wanderers	20	18
16	Brighton and Hove Albion	19	23
17	Leeds United	19	23
18	Burnley	18	16
19	Watford	17	17
20	Norwich City	12	11

Query 6 – Goals Per Team

Business question:

How many goals are scored per team?

SQL:

```
SELECT
    t.TeamName,
    SUM(f.Goals) AS TotalTeamGoals
FROM dw.FactPlayerSeasonStats AS f
JOIN dw.DimTeam AS t ON f.TeamKey = t.TeamKey
GROUP BY t.TeamName
ORDER BY TotalTeamGoals DESC;
```

Result:

Results | Messages

	TeamName	TotalTeamGoals
1	Manchester City	146
2	Liverpool	145
3	Chelsea	119
4	Leicester City	99
5	Tottenham Hotspur	85
6	West Ham United	83
7	Arsenal	76
8	Manchester United	70
9	Brentford	62
10	Crystal Palace	60
11	Southampton	60
12	Aston Villa	57
13	Everton	55
14	Brighton and Hov...	49
15	Leeds United	45
16	Wolverhampton ...	44
17	Newcastle United	41
18	Burnley	37
19	Watford	36
20	Norwich City	30

5. Data Warehouse Creation

The data warehouse schema was implemented using SQL in the **dw** schema. The structure follows the star schema model described earlier, containing five dimension tables and two fact tables. The SQL shown below was taken directly from the implementation scripts used in SQL Server.

5.1 Create DW Schema

```
CREATE SCHEMA dw;
GO
```

5.2 Create Dimension Tables

```
CREATE TABLE dw.DimDate (
    DateKey         INT          NOT NULL PRIMARY KEY,
    FullDate        DATE         NOT NULL,
    SeasonName      NVARCHAR(50) NOT NULL,
    SeasonYearStart INT          NOT NULL,
    SeasonYearEnd   INT          NOT NULL
);

CREATE TABLE dw.DimLeague (
    LeagueKey  INT IDENTITY(1,1) PRIMARY KEY,
    LeagueName NVARCHAR(100) NOT NULL,
    Country    NVARCHAR(50)  NULL,
    SeasonName NVARCHAR(50)  NULL
);

CREATE TABLE dw.DimTeam (
    TeamKey INT IDENTITY(1,1) PRIMARY KEY,
    TeamName NVARCHAR(50) NOT NULL,
    Pos      TINYINT      NULL,
    Pld      TINYINT      NULL,
    W        TINYINT      NULL,
    D        TINYINT      NULL,
    L        TINYINT      NULL,
    GF       TINYINT      NULL,
    GA       TINYINT      NULL,
    GD       SMALLINT     NULL,
    Pts      TINYINT      NULL
);

CREATE TABLE dw.DimPlayer (
    PlayerKey  INT IDENTITY(1,1) PRIMARY KEY,
```

```
    PlayerName       NVARCHAR(100) NOT NULL,
    JerseyNo         TINYINT       NULL,
    FullPositionText NVARCHAR(50)  NULL
);

CREATE TABLE dw.DimPosition (
    PositionKey   INT IDENTITY(1,1) PRIMARY KEY,
    PositionName  NVARCHAR(50) NOT NULL,
    PositionGroup NVARCHAR(50) NULL
);
```

Note*:* Players are matched using PlayerName + JerseyNo, as implemented in the fact table loading process.

5.3 Create Fact Tables

```
CREATE TABLE dw.FactPlayerSeasonStats (
    PlayerSeasonStatsKey INT IDENTITY(1,1) PRIMARY KEY,
    DateKey     INT NOT NULL,
    LeagueKey   INT NOT NULL,
    TeamKey     INT NOT NULL,
    PlayerKey   INT NOT NULL,
    PositionKey INT NOT NULL,
    Goals       INT NOT NULL,
    Appearances INT NOT NULL,
    FOREIGN KEY (DateKey)     REFERENCES dw.DimDate (DateKey),
    FOREIGN KEY (LeagueKey)   REFERENCES dw.DimLeague (LeagueKey),
    FOREIGN KEY (TeamKey)     REFERENCES dw.DimTeam (TeamKey),
    FOREIGN KEY (PlayerKey)   REFERENCES dw.DimPlayer (PlayerKey),
    FOREIGN KEY (PositionKey) REFERENCES dw.DimPosition (PositionKey)
);

CREATE TABLE dw.FactMatchResults (
    MatchKey     INT IDENTITY(1,1) PRIMARY KEY,
    DateKey      INT NOT NULL,
    LeagueKey    INT NOT NULL,
    HomeTeamKey  INT NOT NULL,
    AwayTeamKey  INT NOT NULL,
    HomeGoals    INT NOT NULL,
    AwayGoals    INT NOT NULL,
    FOREIGN KEY (DateKey)      REFERENCES dw.DimDate (DateKey),
    FOREIGN KEY (LeagueKey)    REFERENCES dw.DimLeague (LeagueKey),
    FOREIGN KEY (HomeTeamKey)  REFERENCES dw.DimTeam (TeamKey),
    FOREIGN KEY (AwayTeamKey)  REFERENCES dw.DimTeam (TeamKey) );
```

6. Initial Data Load into DW

6.1 Load Dimension Tables

```sql
INSERT INTO dw.DimDate (DateKey, FullDate, SeasonName,
SeasonYearStart, SeasonYearEnd)
VALUES (20210801, '2021-08-01', 'EPL 2021-2022', 2021, 2022);

INSERT INTO dw.DimLeague (LeagueName, Country, SeasonName)
VALUES ('English Premier League', 'England', 'EPL 2021-2022');

INSERT INTO dw.DimTeam (TeamName, Pos, Pld, W, D, L, GF, GA, GD, Pts)
SELECT Team, Pos, Pld, W, D, L, GF, GA, GD, Pts
FROM dbo.points_table;

INSERT INTO dw.DimPlayer (PlayerName, JerseyNo, FullPositionText)
SELECT DISTINCT Player, JerseyNo, Position
FROM dbo.all_players_stats;

INSERT INTO dw.DimPosition (PositionName, PositionGroup)
SELECT DISTINCT Position, NULL
FROM dbo.all_players_stats;
```

6.2 Load Fact Tables

Fact 1 – Player Season Stats

```sql
INSERT INTO dw.FactPlayerSeasonStats (
    DateKey, LeagueKey, TeamKey, PlayerKey, PositionKey, Goals,
Appearances)
SELECT
    d.DateKey,
    l.LeagueKey,
    t.TeamKey,
    p.PlayerKey,
    pos.PositionKey,
    s.Goals,
    s.Apearances
FROM dbo.all_players_stats s
JOIN dw.DimTeam t          ON t.TeamName  = s.Team
JOIN dw.DimPlayer p        ON p.PlayerName = s.Player
                           AND p.JerseyNo = s.JerseyNo
JOIN dw.DimPosition pos    ON pos.PositionName = s.Position
JOIN dw.DimDate d          ON d.SeasonName = 'EPL 2021-2022'
JOIN dw.DimLeague l        ON l.LeagueName = 'English Premier League';
```

Fact 2 – Match Results

```
INSERT INTO dw.FactMatchResults (
    DateKey, LeagueKey, HomeTeamKey, AwayTeamKey, HomeGoals, AwayGoals
)
SELECT
    d.DateKey,
    l.LeagueKey,
    ht.TeamKey,
    at.TeamKey,
    CAST(LEFT(CONVERT(VARCHAR(8), m.Result, 108), 2) AS INT) AS
HomeGoals,
    CAST(SUBSTRING(CONVERT(VARCHAR(8), m.Result, 108), 4, 2) AS INT)
AS AwayGoals
FROM dbo.all_match_results m
JOIN dw.DimTeam ht ON ht.TeamName = m.HomeTeam
JOIN dw.DimTeam at ON at.TeamName = m.AwayTeam
JOIN dw.DimDate d  ON d.SeasonName = 'EPL 2021-2022'
JOIN dw.DimLeague l ON l.LeagueName = 'English Premier League';
```

7. ETL Process

This section explains how the original CSV files were imported into the **Football_Analysis** database and transformed into the data warehouse star schema.

7.1 Extract

1. The following CSV files were provided:
    - points_table.csv
    - all_match_results.csv
    - all_players_stats.csv
2. Using **SQL Server Management Studio** → **Tasks** → **Import Flat File**, each CSV was imported into the database:
    - dbo.points_table
    - dbo.all_match_results
    - dbo.all_players_stats
3. These tables represent the source (OLTP) layer for the data warehouse.

7.2 Transform

Several steps were required before loading into the dimensional model:

1. Generate surrogate keys

- Identity columns (MatchID, PlayerID) were added to uniquely identify rows in the source tables.
- In the DW, surrogate keys such as TeamKey, PlayerKey, PositionKey, LeagueKey, and DateKey were generated using IDENTITY.

2. Ensure consistency of team names

- Team names in all source tables were cleaned to match points_table.Team.

3. Conversion of match results to goals

- The Result column from dbo.all_match_results was imported as a TIME value (e.g., 02:01:00 instead of 2–1).
- During transformation, it was converted to numeric goal values using:

```
CAST(LEFT(CONVERT(VARCHAR(8), m.Result, 108), 2) AS INT) AS HomeGoals,
CAST(SUBSTRING(CONVERT(VARCHAR(8), m.Result, 108), 4, 2) AS INT) AS
AwayGoals
```

4. Removing duplicates

- DimPlayer and DimPosition were loaded using SELECT DISTINCT.

5. Mapping business keys to surrogate keys

- TeamName → DimTeam.TeamKey
- PlayerName + JerseyNo → DimPlayer.PlayerKey
- PositionName → DimPosition.PositionKey
- SeasonName → DimDate.DateKey
- LeagueName → DimLeague.LeagueKey

7.3 Load

Step 1 – Load dimension tables

- DimDate and DimLeague were manually inserted with one row each.
- DimTeam loaded from points_table
- DimPlayer loaded from all_players_stats
- DimPosition loaded from all_players_stats

Step 2 – Load FactPlayerSeasonStats

Data was loaded from dbo.all_players_stats, joined to dimension tables.

Key measures:

- Goals
- Apearances

Step 3 – Load FactMatchResults

Data was loaded from dbo.all_match_results, and goals were extracted from Result using the conversion logic shown above.

As a result, the original CSV files were successfully transformed into a clean star schema, ready for analytical queries.

8. Automated ETL Process

This section explains the implementation of the automated ETL (Extract, Transform, Load) process. Unlike the initial manual load, this process enables full automation of data loading from CSV files, execution logging, email notifications, and scheduled execution using SQL Server Agent.

8.1 Staging Tables Used for CSV Import

CSV files are first loaded into staging tables for validation before transforming to the final source tables.

```sql
CREATE TABLE dbo.stg_all_match_results (
    [Date] DATE NULL,
    HomeTeam NVARCHAR(50) NULL,
    [Result] TIME(0) NULL,
    AwayTeam NVARCHAR(50) NULL);

CREATE TABLE dbo.stg_all_players_stats (
    Team NVARCHAR(50) NULL,
    JerseyNo NVARCHAR(20) NULL,
    Player NVARCHAR(100) NULL,
    [Position] NVARCHAR(50) NULL,
    Apearances NVARCHAR(20) NULL,
    Substitutions NVARCHAR(20) NULL,
    Goals NVARCHAR(20) NULL,
    Penalties NVARCHAR(20) NULL,
    YellowCards NVARCHAR(20) NULL,
    RedCards NVARCHAR(20) NULL);
```

8.2 ETL Execution Logging Table

```sql
CREATE TABLE dbo.ETL_LoadLog (
    LoadID INT IDENTITY(1,1) PRIMARY KEY,
    LoadStart DATETIME2(0) NOT NULL,
    LoadEnd DATETIME2(0) NULL,
    Status NVARCHAR(20) NOT NULL,   -- 'SUCCESS' or 'FAILED'
    ErrorMessage NVARCHAR(4000) NULL );
```

8.3 Stored Procedure for Automated Daily Load

The following procedure loads CSV data, refreshes the DW, and sends an email notification:

```
CREATE PROCEDURE dbo.usp_DW_DailyLoad_FromCsv
AS
BEGIN TRY
    DECLARE @StartTime DATETIME2(0) = SYSDATETIME();

    INSERT INTO dbo.ETL_LoadLog (LoadStart, Status)
    VALUES (@StartTime, 'STARTED');

    -- 1. Clear staging and source tables
    TRUNCATE TABLE dbo.stg_all_match_results;
    TRUNCATE TABLE dbo.stg_all_players_stats;
    TRUNCATE TABLE dbo.all_match_results;
    TRUNCATE TABLE dbo.all_players_stats;
    DELETE FROM dbo.points_table;

    -- 2. Load CSV files into staging and source
    BULK INSERT dbo.points_table
    FROM 'C:\Users\DEll\Desktop\Year 3\Advanced Database\Assignment
1\points_table.csv'
    WITH (FIRSTROW = 2, FIELDTERMINATOR = ',', ROWTERMINATOR = '\n',
CODEPAGE = '65001');

    BULK INSERT dbo.stg_all_match_results
    FROM 'C:\Users\DEll\Desktop\Year 3\Advanced Database\Assignment
1\all_match_results.csv'
    WITH (FIRSTROW = 2, FIELDTERMINATOR = ',', ROWTERMINATOR = '\n',
CODEPAGE = '65001');

    BULK INSERT dbo.stg_all_players_stats
    FROM 'C:\Users\DEll\Desktop\Year 3\Advanced Database\Assignment
1\all_players_stats.csv'
    WITH (FIRSTROW = 2, FIELDTERMINATOR = ',', ROWTERMINATOR = '\n',
CODEPAGE = '65001');

    -- 3. Move data from staging to OLTP tables (with conversion)
    INSERT INTO dbo.all_match_results ([Date], HomeTeam, [Result],
AwayTeam)
    SELECT [Date], HomeTeam, [Result], AwayTeam
    FROM dbo.stg_all_match_results;

    INSERT INTO dbo.all_players_stats (
```

```sql
        Team, JerseyNo, Player, [Position],
        Apearances, Substitutions, Goals,
        Penalties, YellowCards, RedCards
    )
    SELECT
        Team,
        ISNULL(TRY_CAST(JerseyNo AS INT), 0),
        Player,
        [Position],
        ISNULL(TRY_CAST(Apearances AS INT), 0),
        ISNULL(TRY_CAST(Substitutions AS INT), 0),
        ISNULL(TRY_CAST(Goals AS INT), 0),
        ISNULL(TRY_CAST(Penalties AS INT), 0),
        ISNULL(TRY_CAST(YellowCards AS INT), 0),
        ISNULL(TRY_CAST(RedCards AS INT), 0)
    FROM dbo.stg_all_players_stats;

    -- 4. Refresh Data Warehouse
    EXEC dbo.usp_DW_Reload_FromSource;

    -- 5. Mark success
    UPDATE dbo.ETL_LoadLog
    SET LoadEnd = SYSDATETIME(), Status = 'SUCCESS'
    WHERE LoadID = SCOPE_IDENTITY();

    -- 6. Send success email
    EXEC msdb.dbo.sp_send_dbmail
         @profile_name = 'DW_Mail_Profile',
         @recipients = 'haneens899@gmail.com',
         @subject = 'EPL DW Daily Load - SUCCESS',
         @body = 'The nightly ETL process completed successfully.';
END TRY
BEGIN CATCH
    UPDATE dbo.ETL_LoadLog
    SET LoadEnd = SYSDATETIME(), Status = 'FAILED',
        ErrorMessage = ERROR_MESSAGE()
    WHERE LoadID = SCOPE_IDENTITY();

    EXEC msdb.dbo.sp_send_dbmail
         @profile_name = 'DW_Mail_Profile',
         @recipients = 'haneens899@gmail.com',
         @subject = 'EPL DW Daily Load - FAILED',
         @body = ERROR_MESSAGE();
END CATCH;
```

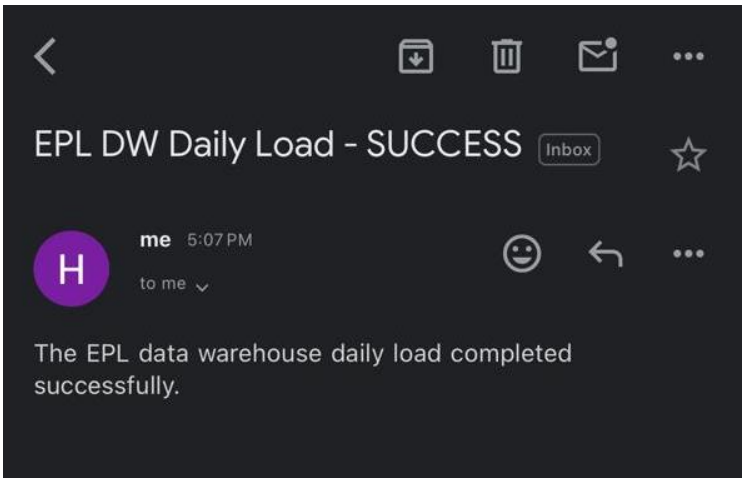
8.4 SQL Agent Job Configuration

To automate execution, a SQL Agent job was scheduled to run daily at 02:00 AM:

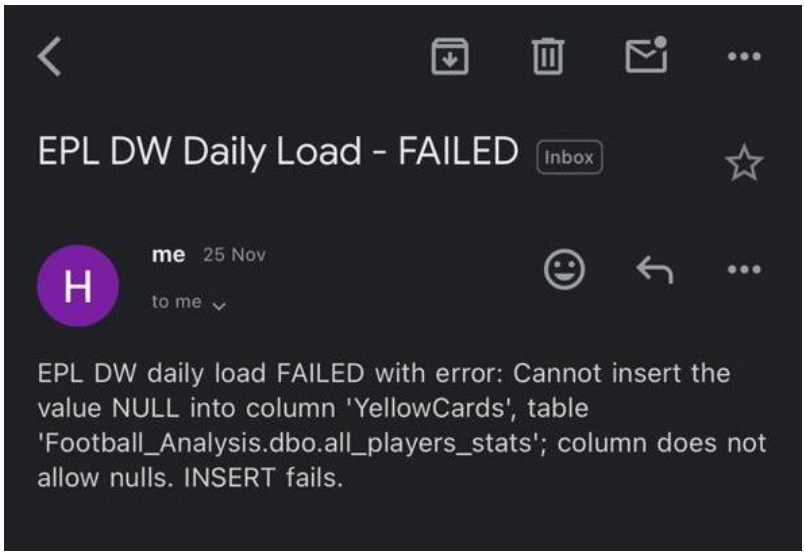
```sql
USE msdb;
GO

EXEC sp_add_job @job_name = N'EPL_DW_Daily_Load';

EXEC sp_add_jobstep
    @job_name = N'EPL_DW_Daily_Load',
    @step_name = N'Run Daily ETL',
    @subsystem = N'TSQL',
    @command = N'EXEC
Football_Analysis.dbo.usp_DW_DailyLoad_FromCsv;',
    @database_name = N'Football_Analysis';

EXEC sp_add_schedule
    @schedule_name = N'Daily_2AM',
    @freq_type = 4,  -- daily
    @freq_interval = 1,
    @active_start_time = 020000;

EXEC sp_attach_schedule @job_name = N'EPL_DW_Daily_Load',
@schedule_name = N'Daily_2AM';

EXEC sp_add_jobserver @job_name = N'EPL_DW_Daily_Load';
```

8.5 Evidence of Successful Execution

1. **Email showing successful ETL execution:**



- Email showing failed ETL execution:



2. **ETL execution log table (ETL_LoadLog) with "SUCCESS" using this query:**

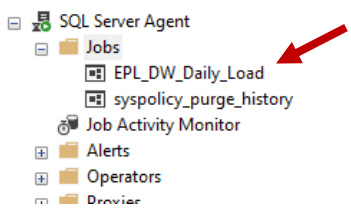
```
SELECT * FROM dbo.ETL_LoadLog ORDER BY LoadID DESC;
```

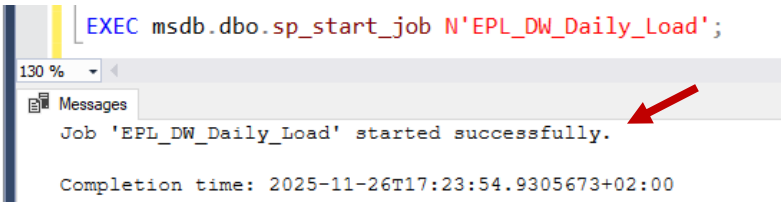
ETL Execution Histroy (ETL_LoadLog):

Results | Messages

	LoadID	LoadStart	LoadEnd	Status	ErrorMessage
1	1002	2025-11-26 17:07:55	2025-11-26 17:07:56	SUCCESS	NULL
2	6	2025-11-25 09:44:01	2025-11-25 09:44:01	SUCCESS	NULL
3	5	2025-11-25 09:41:22	2025-11-25 09:41:23	FAILED	Cannot insert the value NULL into column 'YellowC...
4	4	2025-11-25 09:37:44	2025-11-25 09:37:44	FAILED	Cannot fetch a row from OLE DB provider "BULK" f...
5	3	2025-11-25 09:05:54	2025-11-25 09:05:54	FAILED	Cannot fetch a row from OLE DB provider "BULK" f...

3. **SQL Agent Job Running:**





4. **Stored Procedure Execution Test:**

```
EXEC dbo.usp_DW_DailyLoad_FromCsv;
```

130 %

Messages

```
Mail (Id: 1006) queued.

Completion time: 2025-11-26T17:26:55.1837866+02:00
```

9. Incremental Load & SCD Type 6

This section explains how incremental data loading and the Slowly Changing Dimension (SCD) Type 6 technique are implemented to manage historical changes in team performance over time.

Unlike the initial full reload, this method updates only the changed records and preserves historical values.

9.1 Incremental Load Concept

Instead of refreshing the full data warehouse every time, the incremental load process identifies rows that have changed since the last load (e.g., updated team standings) and only updates those records, which:

- Improves performance
- Retains historical tracking of changes (wins, points, ranking)
- Supports time-based analytics

9.2 Extending DimTeam for SCD Type 6

To allow tracking of historical data while still maintaining current values, new columns were added to **DimTeam**:

```sql
ALTER TABLE dw.DimTeam
ADD ValidFrom DATE DEFAULT '2021-08-01',
    ValidTo   DATE DEFAULT '9999-12-31',
    IsCurrent BIT  DEFAULT 1;
GO

ALTER TABLE dw.DimTeam
ADD CurrentPts TINYINT,
    CurrentPos TINYINT;
GO

UPDATE dw.DimTeam
SET
    ValidFrom = '2021-08-01',
    ValidTo = '9999-12-31',
    IsCurrent = 1,
    CurrentPts = Pts,
    CurrentPos = Pos;
```

Explanation

- **ValidFrom / ValidTo:** Define the time period for each version (Type 2 feature)
- **IsCurrent:** Indicates the active version of the record
- **CurrentPts, CurrentPos:** Store overwritten values for direct use (Type 1 feature)

This combination reflects **SCD Type 6**, which merges Types 1 + 2 with additional current-tracking fields.

9.3 SCD Type 6 Procedure

The following stored procedure automatically detects changes (in team Pos or Pts) and updates the dimension accordingly in three steps:

1. Detects changes
2. Ends old records (ValidTo, IsCurrent = 0)
3. Inserts a new version & updates current data

```sql
CREATE OR ALTER PROCEDURE dbo.usp_SCD_Type6_UpdateDimensions
AS
BEGIN
    DECLARE @LoadDate DATE = CAST(GETDATE() AS DATE);

    -- Detect teams with updated performance
    SELECT
        s.Team AS TeamName,
        s.Pos  AS NewPos,
        s.Pts  AS NewPts
    INTO #Changes
    FROM dbo.points_table s
    JOIN dw.DimTeam t ON s.Team = t.TeamName
    WHERE t.IsCurrent = 1
      AND (s.Pts <> t.Pts OR s.Pos <> t.Pos);

    IF NOT EXISTS (SELECT 1 FROM #Changes)
    BEGIN
        DROP TABLE #Changes;
        RETURN;
    END

    -- Close previous version
    UPDATE t
    SET ValidTo   = DATEADD(DAY, -1, @LoadDate),
        IsCurrent = 0
    FROM dw.DimTeam t
    JOIN #Changes c ON t.TeamName = c.TeamName
    WHERE t.IsCurrent = 1;

    -- Insert new updated version
    INSERT INTO dw.DimTeam (
        TeamName, Pos, Pld, W, D, L, GF, GA, GD, Pts,
        ValidFrom, ValidTo, IsCurrent, CurrentPts, CurrentPos
    )
    SELECT
        c.TeamName, c.NewPos,
        t_old.Pld, t_old.W, t_old.D, t_old.L, t_old.GF, t_old.GA,
t_old.GD,
        c.NewPts,
        @LoadDate, '9999-12-31', 1, c.NewPts, c.NewPos
    FROM #Changes c
    JOIN dw.DimTeam t_old ON t_old.TeamName = c.TeamName
    WHERE t_old.ValidTo = DATEADD(DAY, -1, @LoadDate);
```

```sql
-- Update direct current values (Type 1 aspect)
    UPDATE t
    SET CurrentPts = c.NewPts,
        CurrentPos = c.NewPos
    FROM dw.DimTeam t
    JOIN #Changes c ON t.TeamName = c.TeamName;

    DROP TABLE #Changes;
END;
GO
```

9.4 Example of Change Tracking

Sample Update:

```sql
UPDATE dbo.points_table
SET Pts = Pts + 3, W = W + 1
WHERE Team = 'Liverpool';
```

Then executing the procedure:

```sql
EXEC dbo.usp_SCD_Type6_UpdateDimensions;
```

DimTeam table after SCD Type 6 update showing historical and current versions:

```sql
SELECT * FROM [dw].[DimTeam]
```

130 %

Results | Messages

	TeamKey	TeamName	Pos	Pld	W	D	L	GF	GA	GD	Pts	ValidFrom	ValidTo	IsCurrent	CurrentPts	CurrentPos
1	1081	Arsenal	5	38	22	3	13	61	48	13	69	2021-08-01	9999-12-31	1	69	5
2	1082	Aston Villa	14	38	13	6	19	52	54	-2	45	2021-08-01	9999-12-31	1	45	14
3	1083	Brentford	13	38	13	7	18	48	56	-8	46	2021-08-01	9999-12-31	1	46	13
4	1084	Brighton and Hove Albion	9	38	12	15	11	42	44	-2	51	2021-08-01	9999-12-31	1	51	9
5	1085	Burnley	18	38	7	14	17	34	53	-19	35	2021-08-01	9999-12-31	1	35	18
6	1086	Chelsea	3	38	21	11	6	76	33	43	74	2021-08-01	9999-12-31	1	74	3
7	1087	Crystal Palace	12	38	11	15	12	50	46	4	48	2021-08-01	9999-12-31	1	48	12
8	1088	Everton	16	38	11	6	21	43	66	-23	39	2021-08-01	9999-12-31	1	39	16
9	1089	Leeds United	17	38	9	11	18	42	79	-37	38	2021-08-01	9999-12-31	1	38	17
10	1090	Leicester City	8	38	14	10	14	62	59	3	52	2021-08-01	9999-12-31	1	52	8
11	1091	Liverpool	2	38	28	8	2	94	26	68	92	2021-08-01	2025-11-25	0	95	2
12	1092	Manchester City	1	38	29	6	3	99	26	73	93	2021-08-01	9999-12-31	1	93	1
13	1093	Manchester United	6	38	16	10	12	57	57	0	58	2021-08-01	9999-12-31	1	58	6
14	1094	Newcastle United	11	38	13	10	15	44	62	-18	49	2021-08-01	9999-12-31	1	49	11
15	1095	Norwich City	20	38	5	7	26	23	84	-61	22	2021-08-01	9999-12-31	1	22	20
16	1096	Southampton	15	38	9	13	16	43	67	-24	40	2021-08-01	9999-12-31	1	40	15
17	1097	Tottenham Hotspur	4	38	22	5	11	69	40	29	71	2021-08-01	9999-12-31	1	71	4
18	1098	Watford	19	38	6	5	27	34	77	-43	23	2021-08-01	9999-12-31	1	23	19
19	1099	West Ham United	7	38	16	8	14	60	51	9	56	2021-08-01	9999-12-31	1	56	7
20	1100	Wolverhampton Wande	10	38	15	6	17	38	43	-5	51	2021-08-01	9999-12-31	1	51	10
21	1101	Liverpool	2	38	28	8	2	94	26	68	95	2025-11-26	9999-12-31	1	95	2

The highlighted rows demonstrate how the **previous** version of Liverpool was closed (ValidTo updated and IsCurrent = 0), while the **new updated** record was **inserted** (ValidFrom updated, ValidTo = 9999-12-31, IsCurrent = 1, CurrentPts/CurrentPos updated).

Therefore, The SCD Type 6 solution allows the system to track historical changes in team performance while providing access to current rankings and points in real-time.

10. Conclusion

This project successfully designed and implemented a complete end-to-end **data warehouse** solution for the English Premier League 2021–2022 season. Using dimensional modeling, ETL automation, and SCD Type 6 historical tracking, the system enables accurate and efficient analytical reporting for stakeholders such as coaches, analysts, management, and fans.

The source CSV files were imported, cleaned, standardized, and transformed into a star schema consisting of five dimension tables and two fact tables. This model supports **analytical queries** related to player performance, team standings, match outcomes, and position-based insights.

An **automated ETL** process was implemented using SQL Server stored procedures, staging tables, bulk insert, logging, email notifications, and SQL Agent job scheduling. This automation ensures daily updates, reduces manual intervention, and increases system reliability.

Slowly Changing Dimension Type 6 was implemented for DimTeam to maintain both current and historical team performance records. This hybrid approach supports trend analysis over time while keeping the latest data readily available for reporting.

Overall, the implemented system meets all project objectives, including data integration, transformation, automation, historical tracking, and analytical querying. The solution provides a strong foundation for future enhancements such as dashboard visualization (Power BI/Tableau), support for multiple seasons, multiple leagues, and advanced analytics using machine learning.